

Midterm Exam 1

Time: **15:30-17:30**, November 4, 2025

Note

- Download the *.cpp files from OJ System.
 - Rename each file extension from *.cpp to *.zip and decompress them.
 - Skip the *.h files. The OJ enforces that there must be a *.h file.
- **Submit the Verilog file to OJ immediately once it is done.**
 - You have the responsibility to check if the submission is successful.
 - The latest version submitted to OJ before the deadline will be graded.
 - If you create several modules for any specific problem, merge them into a single Verilog file.
 - The first line of every Verilog file must include your student ID and name as a comment, in the following format:

```
// 112012345 王小明  
module exam_A (...
```
 - DO NOT submit ZIP files, which will be considered as incorrect format.
- Please take the OJ password slip with you when leaving your seat. Do not litter!
- **Follow all submission rules carefully.** Any violation will result in score deductions.
- The submission deadline is **17:30 sharp**. Late submissions will not be accepted.

(I) Verilog Simulation (50%)

Problem A (30%)

In this problem, you will design and implement a **synchronous 4-bit Arithmetic Logic Unit (ALU)** that performs multiple operations on **three 4-bit inputs** based on a 2-bit control signal.

1. To ensure proper timing, **the ALU output must be registered**. That is, you have to add a flip-flop before the output port (i.e., the output will be delayed for one clock cycle).
2. Your design should update its value on the **positive edge** of the clock and be **synchronously reset**.
3. The output must be set to 0 when reset.
4. You must complete the Verilog design in **exam_A.v**
5. You can verify your implementation using the provided testbench (exam_A_tb.v) and pattern file (testcase_A.dat). Hint: See the appendix for instructions on adding these simulation sources.

If your implementation produces incorrect results, error patterns will be displayed during simulation.

```
<ERROR> [pattern 8]: op=00 A=1011 B=0010 C=1111 => out=1110 (ans 0110)
<ERROR> [pattern 20]: op=00 A=1001 B=0101 C=0011 => out=0111 (ans 1111)
<ERROR> [pattern 28]: op=00 A=1011 B=0111 C=1100 => out=1000 (ans 0000)
<ERROR> [pattern 32]: op=00 A=1110 B=0011 C=0001 => out=0100 (ans 1100)
```

During the demo, the TA may provide additional testbench to verify your implementation.

- IO signals and description

Signal Name	I/O	Description
clk	input	Clock (positive-edge triggered)
rst	input	Synchronous active-high reset
A[3:0]	input	Unsigned ALU input
B[3:0]	input	Unsigned ALU input
C[3:0]	input	Unsigned ALU input
op[1:0]	input	ALU control signal
out[3:0]	output	Unsigned ALU output

- Operations

op	Function	Description
2'b00	Conditional Inverter	For each bit i , compare the corresponding bit of A and B. - If $A[i] == B[i]$, output $C[i]$; - Otherwise, output bitwise NOT of $C[i]$
2'b01	Saturating Addition	Perform unsigned addition of $A + B + C$, but limit (i.e., saturate) the result to the maximum 4-bit value (15, or 4'b1111) whenever the result exceeds 15.
2'b10	Bitwise Majority	Perform a bitwise majority vote across the corresponding bits of A, B, and C. That is, for each bit position, output 1 if at least two of the three corresponding inputs ($A[i]$, $B[i]$, $C[i]$) are 1.
2'b11	Left Shift	1. Concatenate inputs in the pattern A, B, A, B to form a 16-bit value 2. Left-shift the 16-bit value by the amount specified by C. 3. Output the most significant 4 bits of the shifted result.

- Examples (you can skip this part if you already understand the description above)

op	A	B	C	result	Briefly Explanation
2'b00	4'b1001	4'b0001	4'b1111	4'b0111	$A[3] = 1, B[3] = 0, C[3] = 1$ $A[3] \neq B[3] \rightarrow \text{result}[3] = \text{inverted } C[3] = 0$
2'b01	4'b1001	4'b0001	4'b1111	4'b1111	$A + B + C = 25 > 15 \rightarrow \text{result} = 15$
2'b01	4'b0001	4'b0011	4'b0001	4'b0101	$A + B + C = 5 \leq 15 \rightarrow \text{result} = 5$
2'b10	4'b1001	4'b1010	4'b0010	4'b1010	$A[3] = 1, B[3] = 1, C[3] = 0$ There are two 1's $\rightarrow \text{result}[3] = 1$
2'b11	4'b0110	4'b0001	4'b0110	4'b0101	Concat ABAB $\rightarrow$ 16'b0110000101100001 shift left by 6 $\rightarrow$ 16'b0101100001000000 Most significant 4 bits $\rightarrow$ 4'b0101

- Grading

Conditional Inverter	7%
Saturating Addition	7%
Bitwise Majority	8%
Left Shift	8%

Problem B (20%)

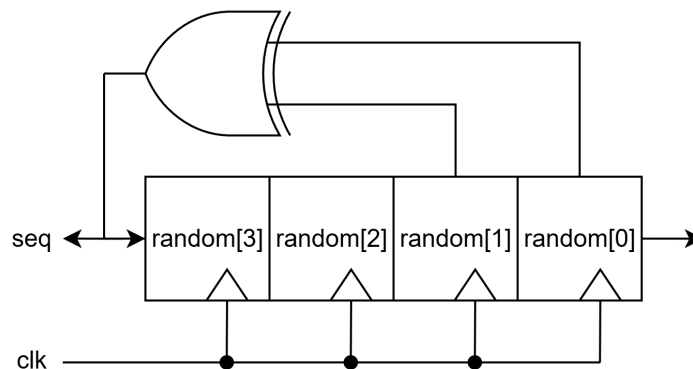
In this problem, you will generate a **pseudo-random** binary sequence using a **Linear Feedback Shift Register (LFSR)**, and implement a **Mealy machine Sequence Detector** that detects a specific bit pattern from the sequence.

- IO signals and description

Signal	I/O	Description
clk	input	Clock (positive-edge triggered)
rst	input	Synchronous active-high reset
seed[3:0]	input	Initial non-zero seed value for the 4-bit LFSR
random[3:0]	output	The 4-bit parallel output from the LFSR
seq	output	The most significant bit of the output from the LFSR. That is, random[3]. This bit is used as the input for the sequence detector.
dec	output	The output of the sequence detector. Outputs 1 whenever the specified pattern is matched; otherwise, outputs 0.

- LFSR module

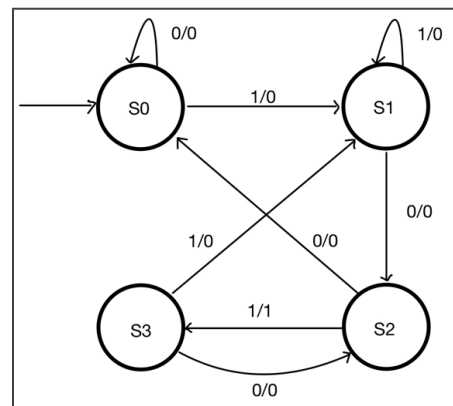
- You will design a **4-bit LFSR**. On each **positive edge** of the clock, the LFSR shifts right by one bit. **The most significant bit (MSB) is updated as the XOR of bit 0 and bit 1.**



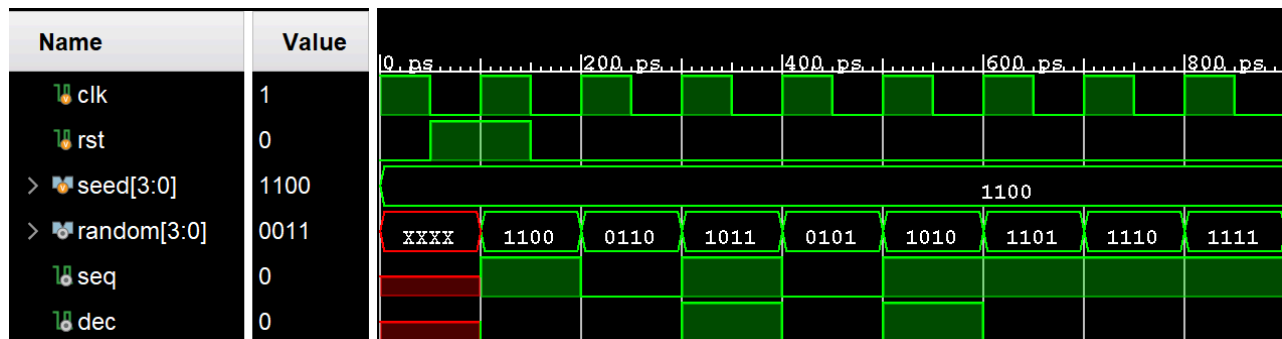
- The parallel output of LFSR should be **initialized to the seed** when **reset**.

- FSM module
- Design a **Mealy Machine Sequence Detector**, which uses the MSB of the LFSR output (seq) as input and produces an output (dec=1) whenever it detects the pattern **1(01)+** in the sequence.
- This specific pattern represents a sequence that **starts with 1**, followed by **one or more repetitions of the pair "01"**. For example, the following sequences match this pattern: 101, 10101, 1010101,
- The Mealy machine that detects this pattern would be as follows:

State	Input	Next state	Output
S0	0	S0	0
S0	1	S1	0
S1	0	S2	0
S1	1	S1	0
S2	0	S0	0
S2	1	S3	1
S3	0	S2	0
S3	1	S1	0



- An example waveform to help illustrate how your LFSR and FSM modules should behave:



Note:

The State of the Mealy Machine Sequence Detector must be **S0** when **reset**.

The output of the Mealy Machine Sequence Detector must be **0** when **reset**.

- Grading:

Sequence Generator (LFSR)	5%
Sequence Detector (FSM)	15%

- You can verify your implementation using the provided testbench (exam_B_tb.v) and pattern file (testcase_B.dat). Hint: See the appendix for instructions on adding these simulation sources.

If your implementation produces incorrect results, error patterns will be displayed during simulation:

```
<ERROR> [pattern 3]: seq=0 (ans 0) => dec=1 (ans 0)
<ERROR> [pattern 18]: seq=0 (ans 0) => dec=1 (ans 0)
<ERROR> [pattern 33]: seq=0 (ans 0) => dec=1 (ans 0)
<ERROR> [pattern 48]: seq=0 (ans 0) => dec=1 (ans 0)
```

During the demo, the TA may provide additional testbench to verify your implementation.

(II) FPGA Implementation (50%) (Problem C)

1. Complete the Verilog template, **exam_C.v**, to implement a **Binary Match Game** on the **FPGA** with a **keyboard**.
2. **DO NOT modify the I/O signals.** You may define I/O signals as either reg or wire, but their names and functionality must remain unchanged.
3. **DO NOT modify the XDC constraint file (exam_C.xdc).** Otherwise, your design will fail to synthesize and result in a ZERO score.
4. There are several modules in the template (**exam_C.v**), including a **Number_Generator for random number generation**, and **SevenSegment for 7-segment display**. You may modify these modules if necessary. However, the submitted code must be able to generate the bit file with the following provided files: **exam_C.xdc**, **clock_divider.v**, **debounce.v**, **onepulse.v**, **KeyboardCtrl.v**, **KeyboardDecoder.v**, and **Ps2Interface.v**.
5. The design must update its value on the **positive edge** of each clock cycle and support **asynchronous active-high reset**. Here is a table showing the function with the I/O connection:

Name	I/O	Pin	Description
clk	input	W5	100MHz clock signal
rst	input	BTNU	Asynchronous active-high reset
restart	input	BTNC	Control signal to restart the game
en	input	BTND	Control signal to enter the Game state
SW [15:0]	input	SW15-SW0	SW6-SW0 control player's bit selections; Others unused;
PS2_CLK	inout	C17	PS/2 keyboard clock
PS2_DATA	inout	B17	PS/2 keyboard data
DIGIT [3:0]	output		Signal to control the 7-segment digits
DISPLAY [6:0]	output		Signal to control the 7-segment encoding
LED [15:0]	output	LD15-LD0	LD6-LD0 show the player's selected bits; LD12-LD7 only active when game is in IDLE state; LD15-LD13 show the player's score;

6. Please refer to the demo video (**DEMO.mp4**) for further details.

7. Brief Description

(1) Game Structure:

Each round begins with a **randomly generated number** (between 0 and 99) shown on the **7-segment display**. The player must **identify which bits in that number's binary representation are 1**. Using the **hardware switches**, the player sets the corresponding bit positions to 1 to indicate the selection. Once finished, the player can **press ENTER** to submit the answer, or **press SPACE** to skip the question and generate a new target number.

(2) Player Control

- Switches (SW6-SW0): Set the corresponding bits to 1 (on = 1; off = 0).
- ENTER (KEY_ENTER): Submit the current answer.
- SPACE (KEY_SPACE): Skip the question and generate a new target number.

(3) Scoring Rules

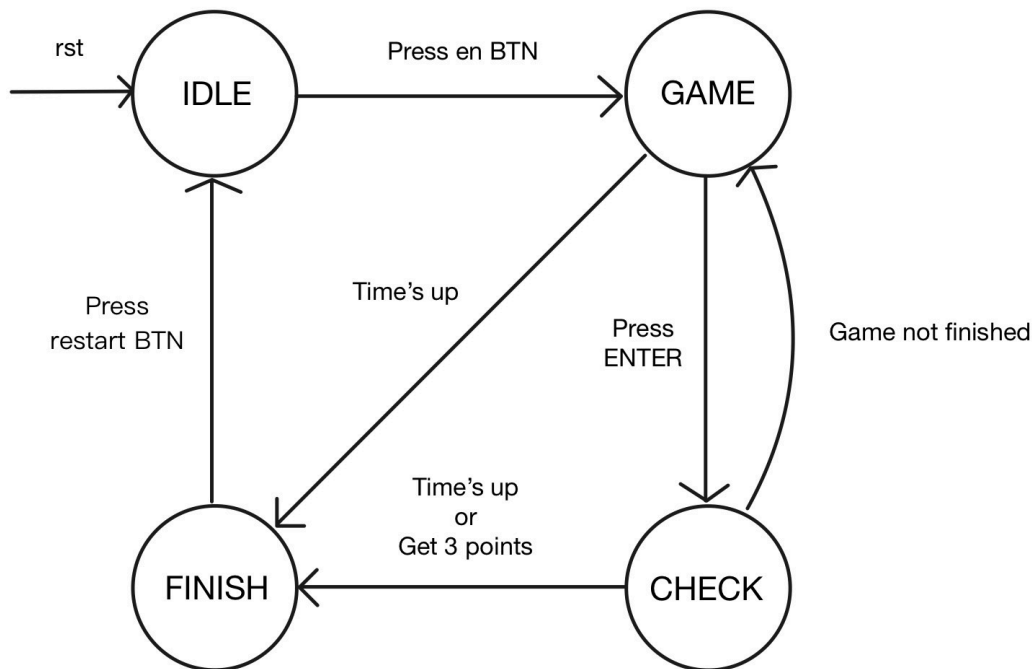
Correct answer: **+1 point**. Incorrect answer or skipped question: **0 point**

(4) Winning Condition

The player **wins the game** if they accumulate **3 points within the time limit of 60 seconds**.

Note: Approximate 1 second by 2^{26} cycles of the 100 MHz clock.

8. The game consists of four states: **IDLE**, **GAME**, **CHECK** and **FINISH**.



9. IDLE state

(1) Pressing the **rst** button at any time will reset the state to the **IDLE** state.

(2) **LED:**

All LEDs (LD15-LD0) are turned on.

7-segment Display:

Displays "60--" as follows:



(3) **Keyboard:**

No input has any effect in this state.

(4) **Switches:**

No input has any effect in this state.

(5) **State Transition:**

Pressing the **en** button transitions the game to the **GAME** state.

10. GAME state

(1) **LED & Switches:**

LD15-LD13: Indicate the player's score and light up sequentially as the score increases from 1 to 3.

LD12-LD7: Turned off.

LD6-LD0: Represent the player's answer in binary form. When any of the rightmost seven switches (**SW7-SW0**) are turned on, the corresponding bits of the player's answer are set to 1.

Note that in the **GAME** state, LD15–LD13 will never all light up simultaneously, since reaching a

score of 3 will trigger a transition from the **CHECK** state to the **FINISH** state. Once in the **CHECK** state, the system does not return to the **GAME** state if the maximum score has been achieved.

Example:

The player already got 1 point, and the currently selected answer is 0110111.

(LD15) ●○○ ○○○○○○ ○●●○●●● (LD0)

The player already got 2 points, and the currently selected answer is 0110111.

(LD15) ●●○ ○○○○○○ ○●●○●●● (LD0)

(2) Keyboard:

ENTER: Submit the answer;

SPACE: Skip the question.

Note: Once skipped, a new number from the Number_Generator will be updated as the target number (0-99).

(3) 7-segment Display:

Left two digits: remaining time in seconds;

Right two digits: the current target number.

The countdown starts at 60. For example, if the target number is 22 and 50 seconds has past since the game began, the 7-segment display should look like this



(4) State Transition:

If the player presses **ENTER**, transition to the **CHECK** state. Otherwise, if the timer counts to 0, transition to the **FINISH** state.

11. CHECK state

(1) LED:

LD15-LD13: Indicate the player's score and light up sequentially as the score increases from 1 to 3.

LD12-LD7: Turned off.

LD6-LD0: Represent the correct binary pattern of the target number.

Example:

The player already got 1 point, and the correct answer is 0110111.

(LD15) ●○○ ○○○○○○ ○●●○●●● (LD0)

The player already got 2 points, and the correct answer is 0110111.

(LD15) ●●○ ○○○○○○ ○●●○●●● (LD0)

The player already got 3 points, and the correct answer is 0110111.

(LD15) ●●● ○○○○○○ ○●●○●●● (LD0)

(2) 7-segment Display:

Same as **GAME** state.

(3) **Keyboard:**

No input has any effect in this state.

(4) **Switches:**

No input has any effect in this state.

(5) **State Transition:**

Stay for 2 seconds. If the player reaches 3 points, transition to the **FINISH** state. Otherwise, go back to the **GAME** state. When going back to the **GAME** state, a new number from the Number_Generator is updated as the target number (0-99).

12. FINISH state(1) **LED:**

All LEDs (LD15-LD0) are turned on.

(2) **7-segment:**

If Player wins, display “UUIn”. Else, display “LoSE”. The display should appear as follows:

 and 

(3) **Keyboard:**

No input has any effect in this state.

(4) **Switches:**

No input has any effect in this state.

(5) **State transition:**

Pressing the **restart** button transitions back to the **IDLE** state.

13. Grading

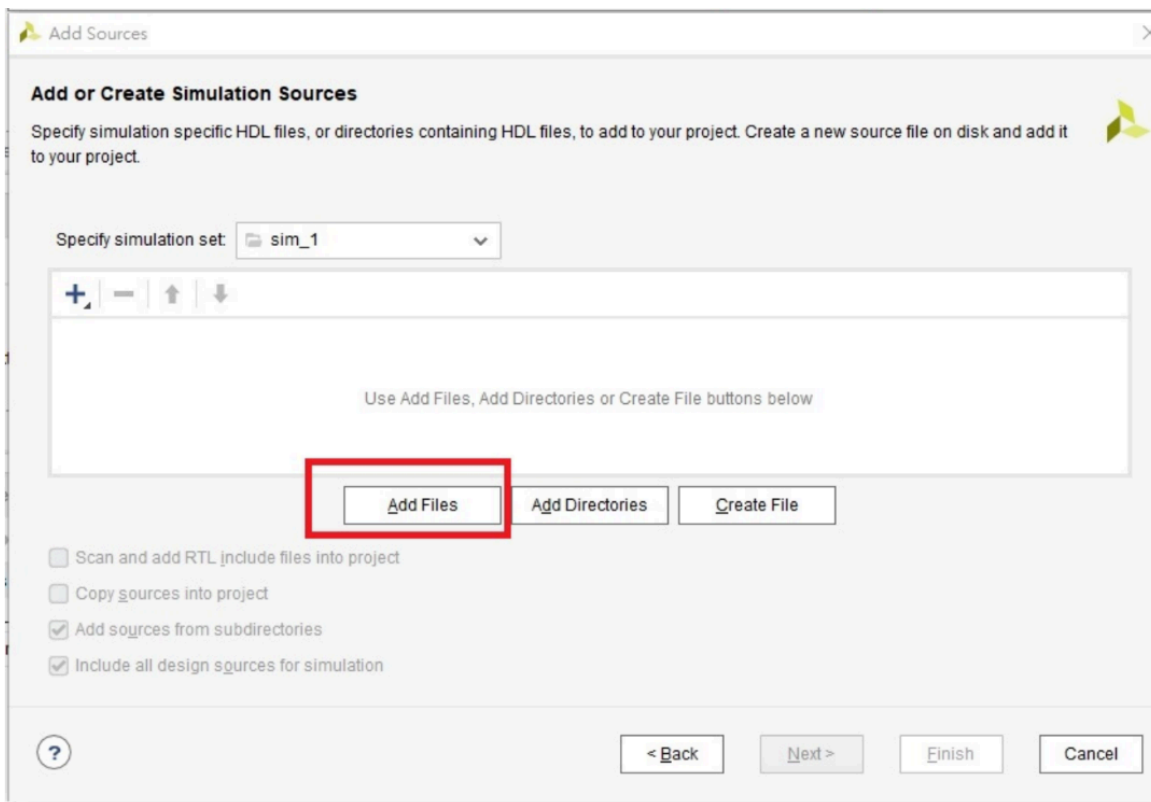
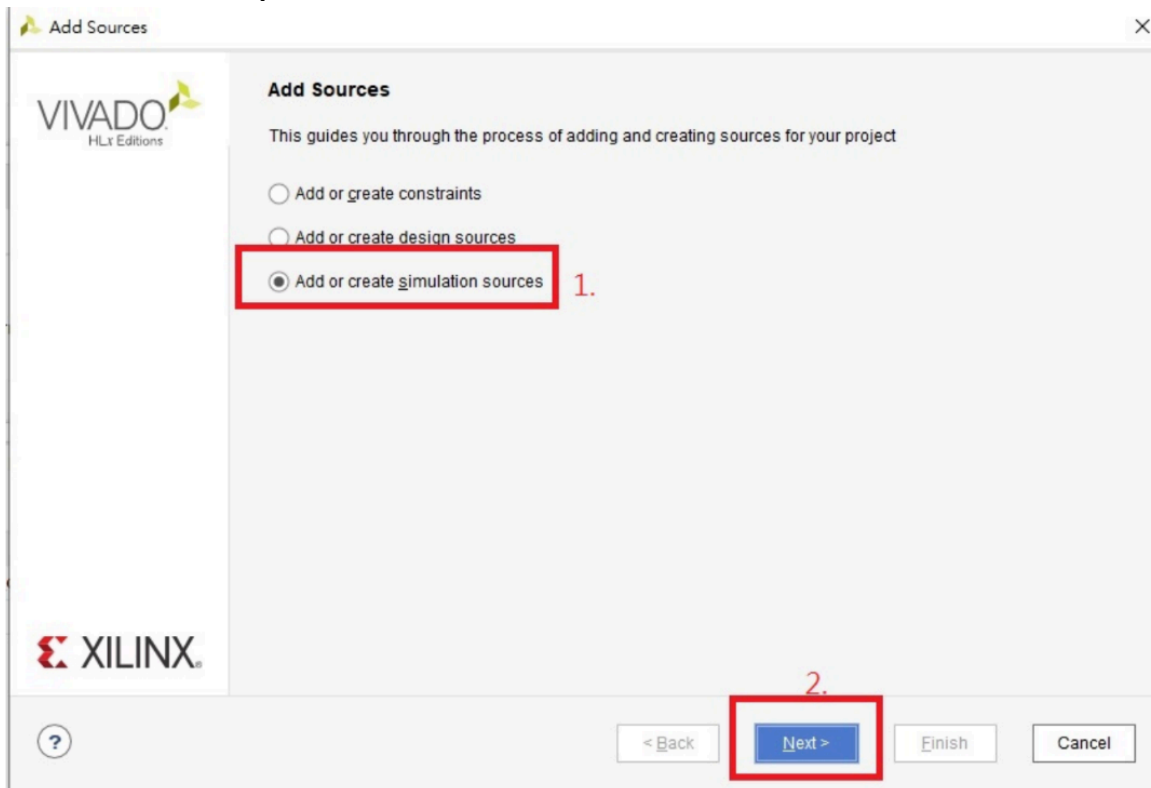
Functions	Score (50%)
IDLE State	6%
GAME State: Keyboard Interactions	10%
GAME State: LED & DISPLAY	10%
CHECK State	15%
FINISH State	6%
Reset	3%

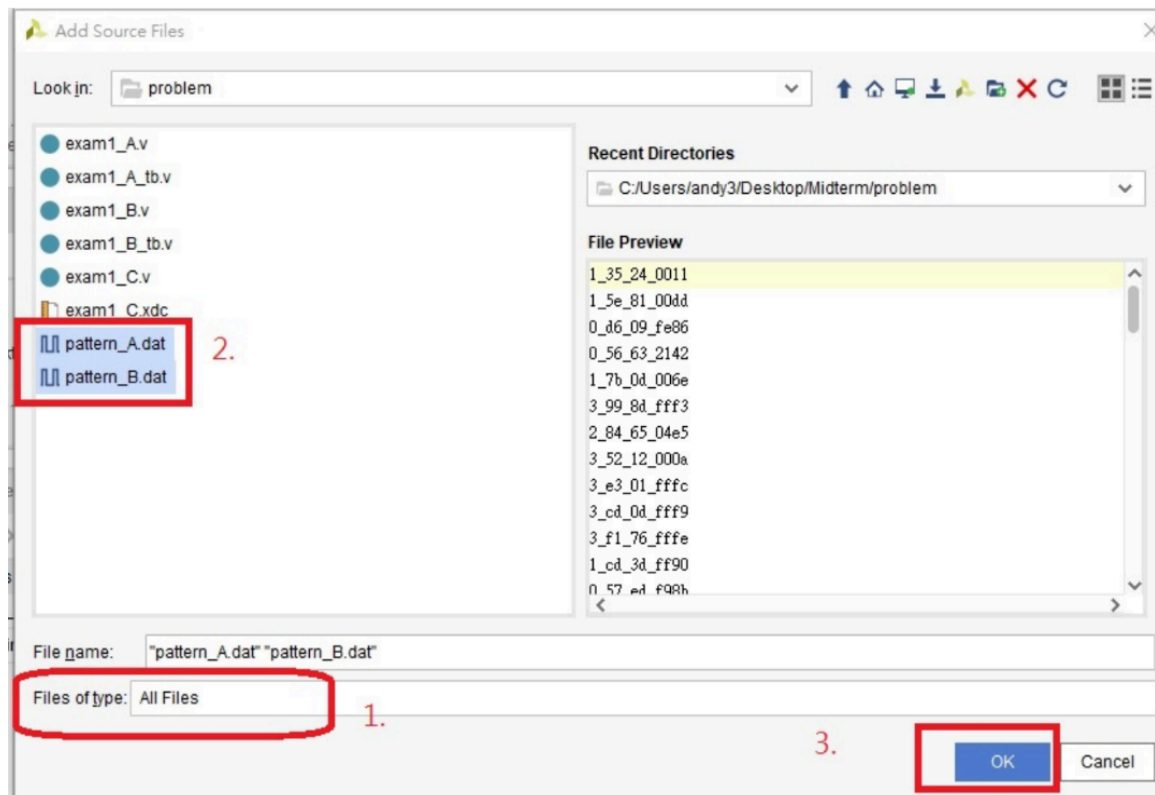
Hint:

Refer to Appendix for keyboard scancodes and binary representations of decimal numbers 0-99.

Appendix

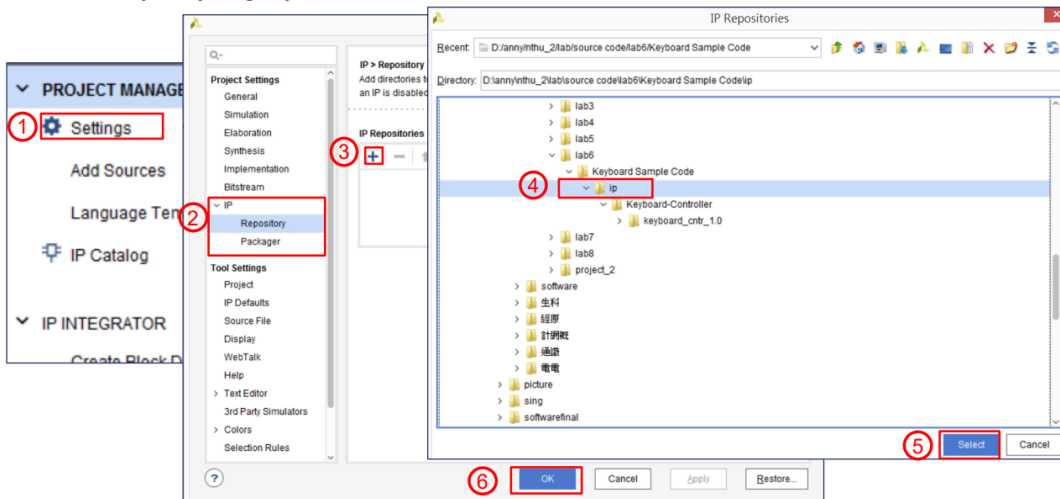
- How to add pattern.dat to the simulation sources:

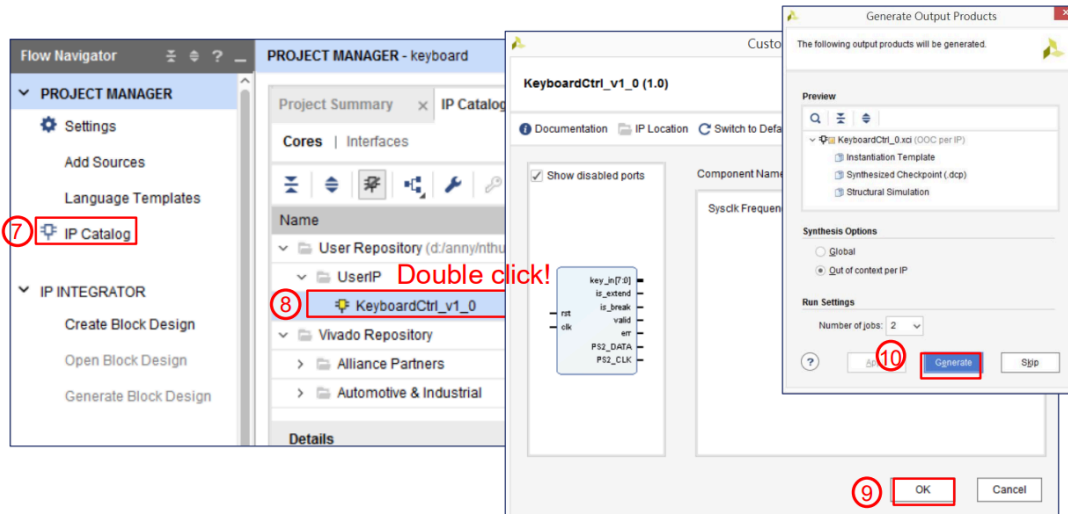




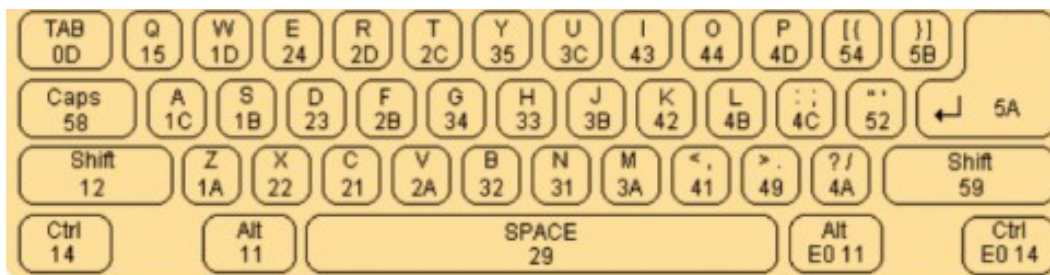
- **How to use keyboard IP:**

*The UI may vary slightly in different versions





- **Keyboard Scancodes:**



- **Binary Representation:**

DEC	BIN	DEC	BIN	DEC	BIN
0	0000000	40	0101000	80	1010000
1	0000001	41	0101001	81	1010001
2	0000010	42	0101010	82	1010010
3	0000011	43	0101011	83	1010011
4	0000100	44	0101100	84	1010100
5	0000101	45	0101101	85	1010101
6	0000110	46	0101110	86	1010110
7	0000111	47	0101111	87	1010111
8	0001000	48	0110000	88	1011000
9	0001001	49	0110001	89	1011001
10	0001010	50	0110010	90	1011010
11	0001011	51	0110011	91	1011011
12	0001100	52	0110100	92	1011100
13	0001101	53	0110101	93	1011101
14	0001110	54	0110110	94	1011110
15	0001111	55	0110111	95	1011111
16	0010000	56	0111000	96	1100000
17	0010001	57	0111001	97	1100001
18	0010010	58	0111010	98	1100010
19	0010011	59	0111011	99	1100011
20	0010100	60	0111100		
21	0010101	61	0111101		
22	0010110	62	0111110		
23	0010111	63	0111111		
24	0011000	64	1000000		
25	0011001	65	1000001		
26	0011010	66	1000010		
27	0011011	67	1000011		
28	0011100	68	1000100		
29	0011101	69	1000101		
30	0011110	70	1000110		
31	0011111	71	1000111		
32	0100000	72	1001000		
33	0100001	73	1001001		
34	0100010	74	1001010		
35	0100011	75	1001011		
36	0100100	76	1001100		
37	0100101	77	1001101		
38	0100110	78	1001110		
39	0100111	79	1001111		